# Follow Specifications (30)

## Core aspects of Specifications

- Using Processes
- Using Pipes for communications (streams in Java and Python)

To a maximum of 30 points, follow the following deductions.

| | |
|---|---|
| Minor mistake: core unaffected | −1 |
| Minor mistake: core not used right | −2 |
| Major mistake: core unaffected | −3 |
| Major mistake: core aspect attempted, but very wrong | −5 |
| Core aspect not used | −15 |

# Pass Tests (20)

Testing for this project cannot be done. Each test is worth 4 points. Give 3 points if there are some minor mistakes. Give 2 points if there are many minor mistakes, or if something important is wrong. Give 1 point if there are two to three important things wrong. Give 0 points if the result is completely wrong or more than three important things are wrong.

# DevLog/Commit History (50)

Since this is the first project, please give a bit of leeway for incorrectly submitted projects. Each submission should be a zip file containing a git repository. If students contact you after posting grades because they lost points for not turning it in correctly, please allow them a chance to turn it in correctly for a regrade. This should only be done if they already have the git repository and they can simply zip it and resubmit. The dates in the commit history should be before the due date.

## The commit history (20)

The commit history (`git log`) should tell a high-level story of the students progress. Mark off up to 5 points if the message do not tell you enough, overall. For instance, "new commit" would be a very bad message (-5 if all message are like this) and "Session 1 start" would be medium (-3 if all messages are like this).

The working tree should be "up to date". A `git status` will tell you. If there are untracked files, they can be ignored unless they are the devlog or source code. The devlog and source code should all be tracked. Deduct 10 points if either is not (for a maximum of 20 points)

If the commit history (or devlog) indicates that the source code or devlog was added later, mark off 5 points. Choose a commit to checkout (if source code/devlog was added late choose one after it was added). Ensure they are following the protocol given the Git/Project Instructions pdf.

# The devlog (30)

The student is assumed to have used multiple sessions to complete the project.

There should be at least 2 entries for each session. The first should discuss the student's thoughts about the project since the last session, and discuss what is planned for this session. The second should be a short reflection on the progress made during the session.

Be flexible when grading. If the above information is all there, but the structure is different that is fine. For instance, there might not be much to reflect on at the end of the session, if the student was adding to the devlog during the session.

There are 5 categories. Give each category a rating as follows. The number in parenthesis is the number of points that rating is worth.

**Failure (0)** The category is missing or is not demonstrated.
**Poor (2)** The category is demonstrated in a vague or superficial way.
**Good (4)** The category is demonstrated in a way specific to the student's work
**Excellent (6)** The category is demonstrated and explained/supported in a way specific to the work

If you are stuck deciding between two ratings, you can give average. For instance if the effort put in is somewhere between Good and Excellent, you can give 5 points.

## Categories

1. The student put effort into the project
2. The student understands processes
3. The student understands pipes and communication between processes
4. The student encountered and overcome problems
   - You can give the points for this if the student demonstrates that no problems were enountered, though very unlikely.
5. The student did not write the entire devlog at once
   - The student wrote the devlog between bouts of coding/design.